

# Ubuntu・OS基礎 自力課題

M1～M7 Mission Guide

Ubuntu Server 24.04 LTS / 公開Lab v1.0.0  
JDU Cyber Security / 2026-09-25

# Ubuntu and OS Foundations — 自力課題 Mission Guide

対象: Mission 1～7 環境: Ubuntu Server 24.04 LTS 進め方: 授業回ではなく、学生ごとの進度でMissionを進める。

この文書は、commandを自分で選ぶ課題M1～M7である。完全なcommand手順が必要な場合は、先に[GUIDED\_PRACTICE.md](GUIDED\_PRACTICE.md)のP0～P6を実施する。

PとMは別のresourceを使う。Pを完了しても、対応するMはPASSにならない。推奨順序はP0 → P1 → M1 → P2 → M2 → … → P6 → M6 → M7である。旧M0は手順付き練習P0へ移した。統合課題M7には対応するPを設けない。

## 最初にUbuntuへ接続する

AWS Academy Learner Labのセットアップを完了してから、AWSコンソールでCloudShellを開く。CloudShellで次のcommandを実行し、自分のUbuntu演習サーバーへ接続する。

```
ssh jdu-ubuntu
```

接続後、Ubuntuで次を実行する。

```
id -un
```

ssm-userと表示されることを確認する。この文書のUbuntu側の操作は、この接続先で実行する。CloudShellで操作する課題では、課題中の指示に従ってexitでCloudShellへ戻る。

## 共通操作

初回構築時に、すべてのMissionは自動で未完成の状態になる。最初にresetする必要はない。

### 実行user

Ubuntuへ接続した直後の管理userはssm-userである。次のcommandで確認する。

```
id -un
```

jdu-checkとjdu-resetは、必ずssm-userで実行する。別のuserで実行すると、ssm-userへ戻る方法を表示して終了する。

演習用userはjduops、jduviewer、jduwriterである。ssm-userから演習用userへ切り替える例を示す。

```
sudo su - jduops
id
```

元のssm-userへ戻る。

```
exit
id -un
```

この演習で切り替えるuserのうち、Passwordなしでsudoを使えるのはssm-userだけである。例えば、jduwriterからjduviewerへ直接切り替えない。一度exitでssm-userへ戻り、そこからsudo su - jduviewerを実行する。

jduapp、jduweb、jdufinal、jduworkerはservice専用userである。Login shellはnologinであり、su-による対話loginは行わない。必要な確認はssm-userからsudo -u USER COMMANDで1 commandだけ実行する。

現在のMissionを確認する。

```
jdu-check M1
```

教員のprogress serverが設定されている場合、現在の結果は自動送信される。全問PASSする前でも送信される。

RESULTはUbuntu上の課題判定である。REPORTはHTTPS送信の成否である。送信に失敗しても、課題のPASS/FAILは変わらない。自分の匿名server IDは次で確認できる。

```
jdu-progress id
```

通信を一時的に行わず、local判定だけを実行する場合は次を使う。

```
jdu-check M1 --no-submit
```

最初からやり直す場合だけ、そのMissionをresetする。

```
jdu-reset M1
```

reset直後は、すべての課題がFAILになる。これは正常である。LabCheckは状態を修正しない。

## M1 Shell, path, file, and text

### 目的

相対pathと絶対pathを区別する。Directoryとfileを作る。Fileをコピーする。Logから必要な行を取り出す。

### 初期状態

~/jdu-lab/m1/inboxに素材がある。case01は未完成であり、stagingに不要なtmp fileがある。

### 課題

1. 次のdirectory treeを作る。

```
~/jdu-lab/m1/case01/  
├── config/  
├── logs/  
└── notes/
```

1. inbox/config/app.confをcase01/config/app.confへコピーする。
2. inbox/logs/incident.logをcase01/logs/incident.logへコピーする。
3. コピーした2 fileの内容を変更しない。
4. case01の中からtmp fileをすべて除く。
5. incident.logからERRORを含む完全な行だけを取り出し、notes/errors.txtへ保存する。行番号を付けない。
6. incident.logの最後の5行を順序を変えず、notes/recent.txtへ保存する。

## 判定

5項目を判定する。Directory、source file、.tmp、errors.txt、recent.txtがそれぞれ判定に対応する。

```
jdu-check M1
```

確認候補: pwd, ls, tree, mkdir, cp, rm, find, grep, tail

## M2 User, group, permission, and sudo

### 目的

User、primary group、supplementary groupを区別する。共有directoryへgroup accessとsetgidを設定する。

### 初期状態

jduops、jduviewer、opsは存在する。初期状態では、jduviewerだけがsupplementary group opsに誤って所属している。jduopsはopsに所属していない。/srv/jdu-shareとREADME.txtのowner、group、modeは未完成である。

### 課題

1. jduopsをsupplementary group opsへ追加する。
2. jduviewerをsupplementary group opsから削除する。
3. /srv/jdu-shareをroot:opsにする。
4. /srv/jdu-shareをmode 2775にする。setgidを使う。world-writableにしない。
5. /srv/jdu-share/README.txtをroot:ops、mode 664にする。
6. ssm-userからsudo su - jduopsで切り替える。共有directoryにfileを作る。新しいfileのgroupがopsになることも確認する。
7. jduviewerがREADME.txtを読めることを確認する。
8. jduviewerが共有directoryにfileを作れないことを確認する。

jduopsでの確認後はexitでssm-userへ戻る。次にsudo su - jduviewerで切り替える。jduviewerでの確認後もexitで戻る。最後にid -unがssm-userであることを確認してからjdu-check M2を実行する。

## 判定

6課題を判定する。Group membership、directory owner/group、directory mode、sample file、group継承、viewer accessが対応する。

```
jdu-check M2
```

確認候補: id, groups, getent passwd, getent group, stat, namei, chown, chmod, usermod, sudo -u

禁止: chmod 777

## M3 Process and package

第8・9章と練習P3の両部分を終わってから取り組む。第10章では、サービスの起動と自動起動を詳しく学ぶ。

## 目的

Serviceとprocessを対応させる。PIDを指定してsignalを送る。Ubuntu packageを調査し、installする。

## 初期状態

jdu-m3-process1.service、jdu-m3-process2.service、jdu-m3-process3.serviceが通常のsystemd unitとして動いている。cmatrixは未installである。初期状態を確認するには、次を使う。

```
systemctl list-units --type=service 'jdu-m3-*'  
systemctl status jdu-m3-process1.service jdu-m3-process2.service jdu-m3-process3.service
```

## 課題A: process2だけを停止する

1. jdu-m3-process2.serviceのMain PIDを調べる。
2. psで同じPIDのprocessを確認する。
3. そのPIDだけへTERMを送る。
4. process2が停止し、process1とprocess3が動いていることを確認する。

広い条件のpkill、killall、signal KILLは使わない。

## 課題B: cmatrixをinstallする

1. Package情報を確認する。
2. aptでcmatrixをinstallする。
3. Commandを実行する。終了はCtrl+Cとする。
4. Packageのversionと、/usr/bin/cmatrixを提供するpackageを確認する。

## 判定

2課題を判定する。課題A全体で1件、課題B全体で1件である。提出fileはない。

```
jdu-check M3
```

確認候補: systemctl status, systemctl show, ps, kill, apt show, apt install, dpkg-query, command -v, dpkg -S

# M4 systemd service

## 目的

Unit fileの設定を読み、serviceのactiveとenabledを区別する。

## 初期状態

教師が用意したjdu-status.serviceはloaded、inactive、disabledである。Unit fileは完成済みである。

## 課題

1. systemctl cat jdu-status.serviceでUser、WorkingDirectory、ExecStartを読む。
2. Unit fileを変更せず、serviceをstartする。
3. Boot時にstartするようenableする。

4. Activeとenabledを別々に確認する。

## 判定

2課題を判定する。変更されていないunitのactiveとenabledを別々に確認する。提出fileはない。

```
jdu-check M4
```

確認候補: `systemctl cat`, `systemctl start`, `systemctl enable`, `systemctl is-active`, `systemctl is-enabled`

## M5 Port, socket, and log

### 目的

Service、process、socket、IP address、port、HTTP、journalを一つの実行状態として関連付ける。

### 初期状態

教師が用意したjdu-web.serviceはloaded、inactive、disabledである。Unit fileとcontent fileは完成済みである。TCP 8081にはlistenerがない。

### 課題

1. Unit fileのUser、ExecStart、address、port、content pathを読む。
2. Unit fileを変更せず、jdu-web.serviceをstartする。
3. 127.0.0.1:8081で通信を待ち受けているsocketを確認する。この状態をlistener（待受socket）という。
4. 待受socketを所有するprocessのPIDと、serviceのMain PIDを比較する。
5. 0.0.0.0:8081や[:,]:8081でlistenしていないことを確認する。
6. http://127.0.0.1:8081/へrequestを送り、HTTP responseを確認する。
7. Service user jduwebがcontent fileを読めることを確認する。ファイルと親directoryのpermissionも確認する。
8. http://127.0.0.1:8081/m5-checkへrequestを送る。
9. 現在のservice起動に対応するjournalでREQUEST path=/m5-checkを確認する。
10. TCP 18081にlistenerがなく、HTTP requestが失敗することを確認する。

## 判定

4課題を判定する。ListenerとPID、HTTPとfile access、学生が送ったrequestのlog、open portとclosed portの比較が対応する。LabCheckが送る/requestは、学生の/m5-checkを代行しない。

```
jdu-check M5
```

## M6 SSH and remote operation

### 目的

CloudShellをlocal、Ubuntuをremoteとして区別する。SSH remote commandとscpのupload、downloadを使う。

## 初期状態

SSH鍵、alias `jdu-ubuntu`、Session Manager tunnelは構築済みである。これらは課題点に含めない。CloudShellとUbuntuのMission fileは存在しない。

## 課題

1. CloudShellで`id -un`、`hostname`、`pwd`を確認する。
2. `ssh jdu-ubuntu`またはSSH remote commandで、Ubuntu側の同じ3項目を確認する。
3. CloudShellの`~/jdu-lab/m6/local-source.txt`へ、次の一行だけを書く。

```
JDU SSH transfer test
```

1. `scp`で、Ubuntuの`~/jdu-lab/m6/upload.txt`へuploadする。
2. SSH remote commandを使い、Ubuntuに`~/jdu-lab/m6/remote-result.txt`を作る。値を手入力しない。

```
REMOTE_USER=<Ubuntuで取得したuser>
REMOTE_HOST=<Ubuntuで取得したhostname>
REMOTE_PATH=<Ubuntuで取得したhome directory>
```

1. `scp`で、Ubuntuの`remote-result.txt`をCloudShellの`~/jdu-lab/m6/downloaded-result.txt`へdownloadする。
2. Upload元と先、download元と先を`sha256sum`で比較する。

## 判定

Ubuntu側は2課題を判定する。UbuntuへSSHで入った状態で実行する。この結果はDashboardのM6 Ubuntuへ送られる。

```
jdu-check M6
```

CloudShell側は4課題を判定する。Ubuntuから`exit`してCloudShellへ戻った後に実行する。この結果はDashboardのM6 CloudShellへ送られる。

```
jdu-check M6
```

DashboardのM6欄は、Ubuntu 2/2、CloudShell 4/4、合計6/6を別々に表示する。M6のMission完了は両方が全件PASSになった時点である。どちらか一方だけを再実行しても、もう一方の最新結果は保持される。

両方で全件PASSにする。秘密鍵を表示、提出、移動しない。SSH service、SSH設定、`authorized_keys`を変更しない。

確認候補: `ssh`, `scp`, `sha256sum`, `id`, `hostname`, `pwd`, `printf`

## M7 Integrated Ubuntu check

### 目的

M2、M4、M5の知識を、新しいserviceへ自力で適用する。新しいcommandは追加しない。

### 初期状態

- ・教師配布の`jdu-final.service`はloaded、inactive、disabledである。

- `/srv/jdu-final`と`index.txt`は存在しない。
- `jduwriter`と`jduviewer`は`finalops`に所属していない。

## 課題

1. `systemctl cat jdu-final.service`で`User`、`WorkingDirectory`、`ExecStart`を読む。
2. `/srv/jdu-final`を`root:finalops`、`mode 2775`で作る。
3. `/srv/jdu-final/index.txt`を作る。内容は`/etc/jdu-lab/final-marker`と同じ一行にする。
4. `index.txt`を`root:finalops`、`mode 664`にする。
5. `jduwriter`を`finalops`へ追加する。`jduviewer`は追加しない。
6. `Writer`が`content`を更新でき、`viewer`が更新できないことを確認する。
7. `Unit file`を変更せず、`service`を`start`して`enable`する。
8. `Service`が`jdufinal`で動き、`127.0.0.1:8090`だけで`listen`することを確認する。
9. `HTTP response`が`status 200`で、指定`marker`を返すことを確認する。
10. `http://127.0.0.1:8090/m7-check`へ`request`を送る。
11. 現在の`service`起動に対応する`journal`で`REQUEST path=/m7-check`を確認する。

`User`を切り替える場合は、`ssm-user`から`sudo su - jduwriter`または`sudo su - jduviewer`を使う。各確認後は`exit`で`ssm-user`へ戻る。最後に`id -un`を確認してから`jdu-check M7`を実行する。

## 判定

6課題を判定する。`File`構成、`writer/viewer`、`service`、`socket`、`HTTP`、`journal`が対応する。`Unit file`、`SSH`設定、`LabCheck`を変更しない。

```
jdu-check M7
```

確認候補: `id`, `getent`, `stat`, `chown`, `chmod`, `systemctl`, `ps`, `sudo ss -lntp`, `curl`, `sudo journalctl`

## 安全上の禁止事項

- `chmod 777`を使わない。
- `/etc/ssh/sshd_config`を変更しない。
- `ssh.service`を停止しない。
- `killall`または広い条件の`pkill`を使わない。
- 秘密鍵を表示、提出、`Git`へ追加しない。
- 外部`host`へ`scan`または`login`試行をしない。